

CSA0307

C04 – Assessment Tool 1

Comparative Analysis

Name B U Sakthi Saailesh

Reg No 192524397

Course Code CSA0307

Course Name Data Structure

Question 1: Collision Resolution Strategies

Hashing maps keys to slots in a table using a hash function, but two different keys can map to the same slot — a collision. Linear probing, quadratic probing, and double hashing are open-addressing strategies that resolve collisions by searching for an alternative slot within the same table, rather than chaining elements externally.

1.1 How each method works

- Linear probing: on collision at slot $h(k)$, check $h(k)+1$, $h(k)+2$, $h(k)+3$, ... (mod table size) until an empty slot is found.
- Quadratic probing: on collision, check $h(k)+1^2$, $h(k)+2^2$, $h(k)+3^2$, ... (mod table size), spacing probes farther apart as attempts increase.
- Double hashing: on collision, the step size itself is computed by a second hash function $h_2(k)$, so probes go to $h(k)+i \cdot h_2(k)$ for $i = 1, 2, 3, \dots$

1.2 Clustering behaviour

Primary clustering is the tendency of consecutive occupied slots to grow into a single large block, which happens because linear probing always tries the very next slot, so any two keys that collide at the same original position end up following the identical probe sequence from that point on. This makes long runs of filled slots increasingly likely to absorb new insertions, slowing the table down as it fills.

Secondary clustering is milder: it occurs when two keys hash to the same initial slot and therefore follow the same probe sequence thereafter, even though the sequence itself spreads out (as in quadratic probing). Keys sharing an initial slot still 'clump' along one path, but different initial slots do not interfere with each other the way they do under linear probing.

Double hashing avoids both forms of clustering because the step size depends on the key itself (via $h_2(k)$). Two keys that collide at the same slot will generally have different second hash values, so they follow different probe sequences from the very first alternate slot onward — this is why double hashing most closely approximates the ideal of uniform (random) probing.

1.3 Comparison table

Aspect	Linear probing	Quadratic probing	Double hashing
Probe sequence	$h(k)+i$	$h(k)+i^2$	$h(k)+i \cdot h_2(k)$
Primary clustering	Severe	None	None
Secondary clustering	N/A (subsumed by primary)	Present, but mild	Virtually eliminated
Avg. successful search	$\approx \frac{1}{2}(1 + 1/(1-\alpha))$	$\approx$ close to double hashing at moderate load;	$\approx -\ln(1-\alpha)/\alpha$ (near-optimal)

Aspect	Linear probing	Quadratic probing	Double hashing
		degrades as α grows	
Avg. unsuccessful search	$\approx \frac{1}{2}(1 + 1/(1-\alpha)^2)$ — grows fastest	Better than linear, worse than double hashing	$\approx 1/(1-\alpha)$ (near-optimal)
Table-size constraint	None	Table size ideally prime; not all slots may be probed	Table size and $h_2(k)$ should be coprime (e.g. prime table size)
Deletion handling	Needs tombstones/lazy deletion	Needs tombstones/lazy deletion	Needs tombstones/lazy deletion

α = load factor (number of entries / table size).

1.4 Cache performance

Linear probing offers the best cache performance of the three, despite its clustering weakness. Because each probe moves to the immediately adjacent memory address, an entire cache line (typically 64 bytes, covering several slots) is loaded on the very first memory access, so subsequent probes are often already in cache. This sequential access pattern is highly predictable and benefits from hardware prefetching.

Quadratic probing jumps by increasing squared offsets, and double hashing jumps by an essentially pseudo-random stride, so both scatter their probes across memory rather than staying local. Each probe is more likely to trigger a fresh cache-line load, giving them weaker cache locality even though double hashing has better probe-count behaviour asymptotically.

In practice this creates a trade-off: linear probing wins on raw cache/memory-access speed at low-to-moderate load factors, while double hashing wins on probe count and predictability at higher load factors where linear probing's clustering cost outweighs its cache advantage.

Question 2: Binary Heap vs Balanced BST for a Scheduler

A scheduler that must repeatedly extract the task with the smallest key (e.g. earliest deadline or highest priority) and must also change the priority of an already-queued task needs a priority-queue structure supporting insert, extract-min, and decrease-key efficiently.

2.1 Time complexity

Operation	Binary heap (array-based)	Balanced BST (AVL / Red-Black)	Notes
Insert	$O(\log n)$	$O(\log n)$	Both bound by tree/heap height

Operation	Binary heap (array-based)	Balanced BST (AVL / Red-Black)	Notes
Extract-min	$O(\log n)$	$O(\log n)$	Heap: remove root, sift down. BST: remove leftmost node, rebalance
Decrease-key	$O(\log n)$ — but only if an external handle/index to the element's array position is maintained	$O(\log n)$ — locate node (needs a pointer/map to it), update, rebalance	Plain array heap has no $O(\log n)$ way to find an arbitrary key without an index map
Find element for update	$O(n)$ without an index map	$O(\log n)$ via BST search	This is the key practical gap

A standard binary heap achieves $O(\log n)$ decrease-key only when it is paired with an auxiliary hash map (or handles) that tracks each element's current array index, since sifting up/down requires knowing the position of the element being updated. Without that auxiliary structure, locating an arbitrary key first costs $O(n)$, making decrease-key effectively linear. A balanced BST does not have this problem: because it is a search structure, locating any key is already $O(\log n)$, so decrease-key is simply a delete-and-reinsert (or in-place key update plus rebalancing), both $O(\log n)$.

2.2 Memory efficiency

The binary heap is more memory-efficient. It is stored as a flat, contiguous array with implicit parent/child relationships computed from index arithmetic (parent = $(i-1)/2$, children = $2i+1$, $2i+2$), so it needs no pointers, no per-node balance metadata, and benefits from excellent cache locality due to contiguous storage.

A balanced BST stores each node as a separate heap-allocated object with (at minimum) left-child, right-child, and parent pointers, plus balance information (a color bit for Red-Black trees, or a height/balance-factor field for AVL trees). This adds substantial per-node overhead — commonly 3–4 extra words per element compared to a heap — and scattered node allocation gives it worse cache behaviour.

2.3 Practical recommendation

If the scheduler mainly performs insert and extract-min with only occasional priority changes, a binary heap (augmented with an index map for decrease-key) is the better choice: it is simpler, more memory-efficient, and cache-friendly. If arbitrary key lookups, frequent decrease-key operations, or in-order traversal of tasks by priority are common requirements, a balanced BST is more natural, since it supports efficient search, predecessor/successor queries, and key updates without needing a separate auxiliary index.